

oldver\testrun3.py

```
1 import os
2 import sys
3 import time
4 import torch
5 import pandas as pd
6 from pathlib import Path
7 from PIL import Image
8
9 from config3 import seed_everything, SEED, CLASS_NAMES, NUM_CLASSES, IMAGE_SIZE
10 from dataset3 import val_transform
11 from model3 import build_model
12
13
14 # — Paths (relative to this script) —————
15 SCRIPT_DIR = Path(__file__).resolve().parent
16 MODEL_PATH = SCRIPT_DIR / "report3.pth" # root-level copy
17 if not MODEL_PATH.exists():
18     MODEL_PATH = SCRIPT_DIR / "outputs" / "report3.pth" # fallback to outputs/
19 DATA_DIR = SCRIPT_DIR / "vir_data_exam"
20 TEST_DIR = DATA_DIR / "test1_vir"
21 OUTPUT_CSV = SCRIPT_DIR / "cls3_ans.csv"
22
23
24 def main():
25     seed_everything(SEED)
26
27     # — 1. Device —————
28     device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
29     print(f"\n Device : {device}")
30     if device.type == "cuda":
31         print(f" GPU : {torch.cuda.get_device_name(0)}")
32
33     # — 2. Load model from checkpoint —————
34     if not MODEL_PATH.exists():
35         print(f" [ERROR] report3.pth not found at {MODEL_PATH}")
36         print(f" Run trainex3.py first, or copy report3.pth here.")
37         sys.exit(1)
38
39     checkpoint = torch.load(MODEL_PATH, map_location=device, weights_only=False)
40     use_se = checkpoint.get("use_se", True)
41     use_aux = checkpoint.get("use_aux", True)
42
43     # Build model with aux=False for inference (aux heads not needed)
44     model = build_model(use_se=use_se, use_aux=False, num_classes=NUM_CLASSES).to(device)
45
46     # Filter out auxiliary classifier weights from checkpoint if present
47     state = {k: v for k, v in checkpoint["model_state"].items()
48             if not k.startswith("aux")}
49     model.load_state_dict(state, strict=False)
50     model.eval()
51
52     print(f" Model : SamGoogLeNetEx3 (SE={use_se})")
53     print(f" Loaded : {MODEL_PATH}")
54     print(f" Epoch : {checkpoint.get('epoch', '?')}")
55     print(f" Val acc: {checkpoint.get('val_acc', '?')}")
56
57     # — 3. Collect test images —————
58     if not TEST_DIR.exists():
59         print(f" [ERROR] Test folder not found: {TEST_DIR}")
60         sys.exit(1)
61
62     image_files = sorted(
63         f for f in os.listdir(TEST_DIR)
64         if f.lower().endswith((".png", ".jpg", ".jpeg"))
65     )
66     print(f" Test images found : {len(image_files)}")
67
68     # — 4. Inference —————
69     results = []
70     t_start = time.time()
```

```

71
72 with torch.no_grad():
73     for fname in image_files:
74         img_path = TEST_DIR / fname
75
76         # RGBA → RGB, apply val transform (clean: resize + crop + norm)
77         image = Image.open(img_path).convert("RGB")
78         tensor = val_transform(image).unsqueeze(0).to(device)
79
80         logits = model(tensor)
81         pred_idx = logits.argmax(dim=1).item()
82         pred_class = CLASS_NAMES[pred_idx]
83
84         # File name without extension (e.g. "image_001")
85         file_stem = Path(fname).stem
86
87         results.append({
88             "filename": file_stem,
89             "prediction": pred_class,
90         })
91
92     elapsed = time.time() - t_start
93     fps = len(image_files) / elapsed if elapsed > 0 else 0
94
95     # — 5. Save CSV —————
96     df = pd.DataFrame(results)
97     df.to_csv(OUTPUT_CSV, index=False)
98
99     print(f"\n Predictions saved : {OUTPUT_CSV}")
100    print(f" Total predictions : {len(df)}")
101    print(f" Inference time : {elapsed:.2f}s ({fps:.1f} images/sec)")
102
103    # Distribution check
104    print("\n Prediction distribution:")
105    for cls_name in CLASS_NAMES:
106        count = len(df[df["prediction"] == cls_name])
107        pct = count / len(df) * 100
108        print(f" {cls_name} : {count:>3d} ({pct:.1f}%)")
109
110    print(f"\n Done\n")
111
112
113 if __name__ == "__main__":
114     main()

```